Strategic Path Forward: Resolving Temporal Paradoxes in Dispatch Packet Drafting Through Certified State Generation

1. Constitutional Context and Systemic Equilibrium

The operational environment under analysis operates within a rigorously defined governance framework encapsulated by the Policy of Truth (PoT.md). Within this architecture, the fundamental preamble dictates that the system does not request compliance; rather, it structurally prevents any action that violates policy.[1] This doctrine establishes a deterministic state machine where human operators, Integrator (Lead AI) agents, and Contractor (Guest AI) agents must interact within strictly defined physical laws, behavioral standards, and execution ledgers. The highest operational standard within this environment is the absolute alignment between canonical documentation, repository state, and execution reality. Deception, hallucination, or incoherence within this system are categorized not merely as errors, but as inherently destabilizing events that trigger immediate system failure states.[1]

Recent forensic analysis of the system's operational behavior reveals a critical epistemic contradiction occurring during the drafting and execution of Dispatch Packets (DP). Specifically, the legacy workflow demands that the Integrator AI generate complete, deterministic closing metadata—including the final technical manifest, file diff outputs, and final squash stubs—during the drafting phase via the ops/bin/draft binary. This phase inherently precedes the Contractor AI's actual execution of the assigned work.[1]

This sequence enforces a temporal paradox: it requires a probabilistic entity (the Integrator) to generate an absolutely certain manifest of a future state.[1] When execution outcomes are unknowable or contingent upon exploratory findings, the Integrator is forced into a dilemma. It must either halt operations entirely due to missing information, or it must fabricate (hallucinate) future repository states to satisfy the structural demands of the DP template and the subsequent preflight linting gates.[1]

This predictive generation stands in direct violation of the "Generation Mandate" outlined in PoT.md Section 4.2, which explicitly dictates that verification outputs must be generated, not pre-filled or assumed, and that the manual or artificial fabrication of receipts is strictly prohibited.[1] Furthermore, the fabrication of paths violates the absolute literalism required by the Behavioral Logic Standard.[1]

To restore system equilibrium and eliminate the conditions that mandate AI hallucination, a

single, unified strategic path forward must be implemented. This report synthesizes the analytical findings regarding the structural malformations in DP generation and formulates a comprehensive architectural transition from a model of "Authored Future State" to a deterministic model of "Certified State Generation," grounded in Correct-by-Construction (CbC) methodologies.[1]

2. Epistemic Failures in the Predictive Drafting Model

The structural flaw within the repository workflow is not an intelligence failure on the part of the Integrator or Contractor agents. Advanced large language models operate on probabilistic token generation; they cannot perfectly simulate the execution of a bash script or a git diff command in an isolated runtime environment without access to the actual execution output. The failure is, therefore, a mechanistic failure of the TASK.md container contract and the ops/src/surfaces/dp.md.tpl canonical template.[1]

2.1 The Generation Mandate Contradiction

According to the physical laws of the repository, all state changes must be verifiable and deterministic. The repository separates the execution environment strictly under the Filing Doctrine: projects/ for payload code, ops/ for runtime binaries, docs/ for rationale, and storage/ for execution receipts and handoff artifacts.[1] Under the legacy protocol, the Integrator generates the DP structure utilizing ./ops/bin/draft and populates the Mandatory Closing Block directly into the TASK.md surface.[1]

Because the Mandatory Closing Block historically included the "Extended Technical Manifest" and the "Final Squash Stub," the Integrator was mandated to declare exactly which files would be altered before the Contractor began the task.[1] If a task involves any degree of investigation—for example, assessing whether a target shell script possesses the appropriate hook architecture before modifying it—the final list of files to be modified remains in a quantum state until the investigation concludes. By forcing the Integrator to provide a static manifest during the initial draft execution, the system forces the AI to guess.[1]

If the Integrator guesses incorrectly, the resulting mismatch between the declared manifest and the actual git diff --name-only generated at the end of the run triggers a "Canon Drift" system failure state.[1] This results in a rejected pull request and a mandatory work stoppage. Conversely, if the Integrator recognizes the impossibility of the prediction and leaves the fields blank, the tools/lint/dp.sh preflight gate immediately fails the packet before work can commence.[1] The AI is trapped between violating the Generation Mandate by guessing, or triggering an Ambiguity failure by omitting the data.

2.2 The Prohibition of If/Then Execution Logic

A secondary malformation occurs when the Integrator attempts to bypass the temporal

paradox by embedding conditional "If/Then" logic within the DP Execution Plan (Section 3.4 of the DP structure).[1] For instance, an Integrator might draft instructions directing the Contractor to "Check file X; if condition Y is met, modify file Z, otherwise modify file W."

This formulation fundamentally violates the absolute literalism and determinism required by PoT.md Section 4.2.[1] A Contractor executing conditional logic modifies the repository state in ways the Integrator could not explicitly authorize within the strict allowlist boundary (storage/dp/active/allowlist.txt). The allowlist is a hard programmatic gate; every tracked mutation must be registered within this text file before execution begins.[1] Conditional execution logic invariably leads to an unauthorized file touch, which is immediately flagged by tools/lint/integrity.sh or tools/lint/task.sh, triggering a process kill.[1] The system's rules require that instructions be entirely immune to multiple interpretations.[1]

2.3 Analysis of Mixed Packets

The root cause of these paradoxes is frequently the drafting of "mixed" packets.[1] A mixed packet combines deterministic implementation tasks with exploratory or investigative tasks within the same Dispatch Packet. Because the specific file outputs or necessary modifications of an exploration are entirely unknown at the drafting stage, mixing them with deterministic implementation breaks the allowlist stricture and forces the Integrator into predictive hallucination.[1] The architectural solution requires addressing both the template structure and the semantic categorization of the tasks themselves.

3. The Correct-by-Construction (CbC) Paradigm Applied to Governance

To resolve the predictive paradox, the system must adopt a protocol grounded deeply in the Correct-by-Construction (CbC) design discipline.[2] CbC is a development methodology that prioritizes mathematical modeling and structural enforcement prior to coding, balancing design robustness with physical constraints.[2]

3.1 Principles of CbC in Deterministic Systems

In advanced systems engineering, the primary objective of CbC is to eliminate defects as early as possible, typically before developers enter the register-transfer level (RTL) design or write testbenches in hardware verification languages.[2] For example, bridge designers create accurate physical models to ensure load-bearing correctness before laying a foundation; similarly, software models are used to reason about proposals to guarantee that required functionality is delivered and correct behavior is exhibited.[2] Testing in a CbC environment is still performed, but its role shifts from finding bugs to validating that the correct-by-construction process was adhered to.[2]

When integrating real-time correct-by-construction methodology, developers reap massive optimization benefits by reducing cycle time and cost, significantly improving product quality, and allowing for real-time product feedback.[4] Furthermore, providing a partial specification, proving properties of that partial specification, and then filling in the rest of the specification in a way that preserves the proven properties is a hallmark of CbC in protocol design.[3]

3.2 Translating CbC to Repository Governance

Within the repository, CbC requires transitioning from post-facto detection (e.g., relying on a continuous integration pipeline to catch a hallucinated file path) to structural prevention at the point of origin.[1] As dictated by PoT.md Section 4.2, operational directives demand that before adding enforcement automation, structural prevention must be evaluated and implemented if viable.[1] The full procedure is outlined in docs/DESIGN.md, which mandates that tooling must not merely detect bad behavior but must render bad behavior physically impossible to express.[1]

The legacy DP template structure relies on the Integrator's adherence to rules through complex prompting, asking the AI to somehow synthesize a future state perfectly. A CbC approach physically removes the static closing block fields from the drafting phase template (ops/src/surfaces/dp.md.tpl) altogether, making it impossible for the Integrator to hallucinate the data during the draft step.[1] If the slots do not exist in the template, the AI cannot fill them with probabilistic guesses.

Enforcement Topology	Paradigm	Mechanism	Outcome
Legacy Predictive	Post-Facto Detection	tools/lint/dp.sh scans for hallucinated paths *after* generation.	High failure rate; forces AI to guess; violates Generation Mandate.
Correct-by-Construction	Structural Prevention	ops/bin/draft utilizes a template with no closing block slots.	Zero hallucination risk; AI focuses entirely on execution steps.

By redefining the dp.md.tpl structure and shifting the locus of metadata generation to the closeout phase, the system aligns perfectly with CbC principles, ensuring that the final output is mathematically bound to the physical file diffs of the repository.

4. Synthesis of Analytical Findings

The resolution of the drafting paradox requires synthesizing observations regarding how advanced language models interact with the rigid boundaries of the Stela platform.[1] The integration of the various AI-generated plans reveals distinct, non-negotiable requirements for correcting the workflow protocol.

4.1 Typological Clarification Requirements

The core issue stems from "mixed" packets.[1] Because the file output of an exploration is unknown, mixing it with implementation breaks the allowlist stricture. The workflow must formally recognize two distinct categories of Dispatch Packets to maintain structural integrity.[1] Investigative operations must be fundamentally decoupled from operations that alter code payloads. If an outcome implies branching possibilities, it cannot exist in the same packet as a definitive file mutation.[1]

4.2 Decoupling Output from Intent

The metadata defining the final result of a task (the Mandatory Closing Block) must be entirely decoupled from the metadata defining the intent of a task (the initial DP contract).[1] The DP contract, housed in TASK.md, should only outline the Operator authorization, the scope boundaries, the file allowlist, and the sequential execution steps.[1] The resulting artifacts, specifically the technical manifest and the execution proof, must be relocated to a separate, asynchronous execution phase.[1]

4.3 Elevating the Certifier

The repository utilizes the ops/bin/certify binary to execute receipt commands, validate integrity, and generate the RESULTS artifact.[1] The analysis demonstrates that this binary must be substantially enhanced to become the sole truth generator for the entire system state, performing Just-In-Time (JIT) metadata generation at the precise moment of execution closeout.[1] The certify binary must move from being a simple receipt formatter to a definitive state-capture engine.

5. The Singular Strategic Path Forward

To achieve absolute system equilibrium and eliminate temporal paradoxes in drafting, the governance system must undergo a phased structural reorganization. This Strategic Path Forward establishes "Certified State Generation" as the singular mechanism for resolving DP execution.[1] This implementation must be executed sequentially to ensure that no existing PoT.md constraints are violated during the transition.

5.1 Phase 0: Immediate Operational Directive (Stopping the Bleeding)

Before code or template architectural changes are finalized, immediate behavioral modifications must be imposed upon the Integrator and Operator routines to prevent the

introduction of further malformed packets into the storage/dp/intake/ staging environment.[1] This requires manual enforcement by the Operator until the systemic structures are updated.

1. **Cessation of Mixed Packets**: The system must immediately halt the generation of Dispatch Packets that combine deterministic implementation with exploratory investigation.[1] If an Integrator proposes a mixed plan, the Operator must reject it and instruct a rewrite.
2. **The "Investigate, Decide, Implement" Pipeline**: Any proposed task requiring a viability check that could theoretically alter the final target file set must be ruthlessly separated. The workflow becomes a minimum of two distinct DPs: an investigation packet (yielding a specific outcome), followed by an Operator decision review, followed by an implementation packet.[1]
3. **Strict Stoppage on Ambiguity**: As mandated by PoT.md Section 2, instructions capable of multiple interpretations require a hard STOP.[1] If an Integrator is asked to draft a plan with branching outcomes, the drafting behavior must output a split recommendation rather than generating placeholders or conditional logic.[1]

5.2 Phase 1: Typological Split as a First-Class Protocol

To enforce the CbC design discipline structurally, the system must formally define and enforce the semantic boundaries of Dispatch Packets.[1] The schema of TASK.md and the validation logic within tools/lint/task.sh must be updated to recognize and police two mutually exclusive DP topologies.[1]

5.2.1 Investigative Dispatch Packets

An Investigative DP is defined as a workflow designed exclusively to discover information, assess technical viability, or audit an existing repository state.[1]

- **Mutation Constraint**: The only permitted repository mutation for an Investigative DP is the creation of a single Decision Record located strictly within the archives/decisions/ directory.[1]
- **Path Determinism**: The exact file path of this Decision Record must be known prior to execution and declared in the allowlist.
- **Prohibitions**: There are to be absolutely no conditional file deletions, no speculative pull request metadata generation, and no code payload modifications.[1] The Execution Plan may only contain commands that read state (e.g., grep, cat, ls) and write to the designated Decision Record.

5.2.2 Deterministic Dispatch Packets

A Deterministic DP is defined as a workflow designed to enact approved, mathematically certain modifications to the repository.[1]

- **Mutation Constraint**: Modifications are strictly bound to a pre-authorized file set

enumerated within storage/dp/active/allowlist.txt.[1]
- **Execution Determinism**: The Execution Plan must consist of linear, imperative instructions.
- **Prohibitions**: Any branching logic, conditional exploration, or deviations from the static allowlist are forbidden. If a worker encounters an unexpected state, they must halt execution rather than executing an alternate branch.[1]

By establishing this typological split as a foundational protocol, the Integrator is never placed in a position where it must guess the downstream effects of a Dispatch Packet.[1] The output of an Investigative DP is always exactly one Decision Record file.[1] The output of a Deterministic DP is always a fixed array of modifications whose paths are perfectly known in advance.

5.3 Phase 2: Decoupling the Closing Block and Certify-Time Generation

Phase 2 addresses the structural mechanism of the temporal paradox. The DP template (ops/src/surfaces/dp.md.tpl) currently forces the Integrator to define the Mandatory Closing Block during the drafting stage.[1] This directly causes the hallucination issue and must be structurally eradicated.

5.3.1 Relocation of the Closing Block to an Execution Sidecar

The static closing block fields—specifically the Primary Commit Header, Pull Request Title, Pull Request Description, Final Squash Stub, and Review Conversation Starter—must be stripped entirely from the drafting template.[1] The responsibility for capturing these closing details must be shifted from the initial drafting phase to the active execution phase.

These fields will now be authored within a dedicated, persistent sidecar file: storage/handoff/CLOSING-DP-OPS-XXXX.md.[1] This sidecar is maintained by the Contractor AI (or the human Operator) during the actual execution of the task.[1] Because this sidecar is populated while the work is being performed or immediately after its completion, the true state of the work is observable and unambiguous, completely eliminating the need for predictive guessing.

The CLOSING-*.md sidecar must adhere to the strict stylistic guidelines enforced by tools/lint/style.sh, including the requirement that the Conversation Starter must end in a question mark, and that the PR Description must contain at least one valid markdown construct.[1]

5.3.2 Enhancing ops/bin/certify as the Absolute Truth Generator

The ops/bin/certify binary currently serves to execute receipt commands and generate a basic RESULTS artifact.[1] It must be significantly upgraded to serve as the absolute mechanism for Just-In-Time (JIT) metadata generation.[1] Manual assembly of receipt content remains strictly

prohibited, as dictated by PoT.md Section 4.2.[1]

Upon task completion, the Operator initiates the finalization protocol by executing the certification command: ./ops/bin/certify --dp=DP-OPS-XXXX --out=auto.[1]

The enhanced certifier will perform the following deterministic, reality-based operations:

1. **Sidecar Ingestion**: The binary will actively read the storage/handoff/CLOSING-DP-OPS-XXXX.md sidecar to retrieve the narrative fields authored during execution.[1] If the sidecar is missing or malformed, the binary will trigger a hard stop.
2. **Verification Check Execution**: The binary will run all mandatory receipt commands listed in the DP's Section 3.4.5.[1] Crucially, if any command involves process substitution (e.g., $(pwd), $(git log)), the certifier must reject it and fail, enforcing the requirement for literal values only, ensuring total reproducibility.[1]
3. **Dynamic Manifest Generation**: Rather than reading a pre-filled list of modified files, the certifier will execute git diff --name-only against the Base HEAD at the exact moment of certification.[1] The output of this command structurally becomes the immutable "Extended Technical Manifest." This mathematically guarantees that the manifest perfectly matches the actual repository reality, flawlessly fulfilling the CbC mandate and entirely eliminating AI hallucination of file paths.[1]
4. **Linguistic and Structural Gates**: The certifier will programmatically invoke tools/lint/style.sh against the ingested sidecar to enforce the Linguistic Precision mandate.[1] This ensures no contractions exist, verifies the required heading structures, and confirms that the dynamically generated manifest contains valid file paths only.[1]
5. **Receipt Rendering**: Finally, the certifier will compile the ingested narratives, the dynamically generated manifest, the output of the verification commands, and the DP metadata into the final RESULTS receipt using the ops/src/surfaces/results.md.tpl template.[1]

This JIT generation model transforms the RESULTS document from a reflection of the Integrator's intent into an unimpeachable record of the repository's physical state.

5.4 Phase 3: Prompt and Gate Hardening

To ensure the new architectural boundaries are respected by all operating agents, the Integrator prompt stances and the automated gate workflows must be rigidly hardened to reflect the new paradigms.[1]

5.4.1 Hardening the Integrator Prompt Stance (E-PROMPT-03)

The instructional boundaries for the Integrator (Lead AI), currently defined within docs/ops/prompts/e-prompt-03.md, govern how the AI approaches drafting.[1] This prompt must be updated to install the following immutable behavioral logic constraints:

- **Synthesis Ban**: The Integrator is explicitly forbidden from synthesizing or predicting technical manifests for future execution states.[1] It must be instructed to rely entirely on the execution-phase sidecar and the certifier binary for output generation.
- **Path Reference Prohibition**: The Integrator must never reference session-bound, ephemeral paths located within storage/handoff/ (e.g., OPEN artifacts, dump texts) or var/tmp/ when authoring permanent ledger files such as SoP.md, PoW.md, or the permanent DP execution plan.[1] These ephemeral files vanish after the closeout cycle, and referencing them creates dead pointers in the historical record.
- **Non-Deterministic Logic Prohibition**: The use of conditional "If/Then" logic within the Execution Plan section (Section 3.4 of the DP) is strictly prohibited.[1] Every execution step must be a definitive, unconditional instruction (e.g., "Step A: Run script X. Step B: Save output to file Y").[1] If the Integrator cannot provide a definitive step, it must default to drafting an Investigative DP.
- **Addendum Authority Enforcement**: The prompt must reiterate that an Integrator produces Addendum recommendations as fully rendered artifacts (using ops/src/surfaces/addendum.md.tpl) only *after* a post-work audit identifies scope expansion requirements. The AI must never pre-author Addendums during the drafting phase.[1]

5.4.2 Hardening Automated Linters and Execution Gates

The shell scripts responsible for policing the repository—the "Automation" pillar of the enforcement mandate—must be updated to align with the new certification methodology and support the CbC architecture.[1]

- **tools/lint/dp.sh**: This script enforces DP transaction rules.[1] It must be modified to verify the existence and basic formatting of the storage/handoff/CLOSING-DP-OPS-XXXX.md sidecar during the execution phase.[1] It must rigidly enforce that the Freshness Stamp field format is strictly YYYY-MM-DD and reject any trace tokens, ISO-8601 timestamps, or natural language.[1] Furthermore, it must scan the RECEIPT_COMMANDS slot during the preflight check; if any command substitution syntax (e.g., $(pwd)) is detected, the linter must fail the DP, ensuring compatibility with the certifier's literal replay engine.[1]
- **tools/lint/integrity.sh**: This script must be enhanced to perform a strict reconciliation check: it must verify that the dynamically generated Extended Technical Manifest produced by the git diff --name-only command precisely matches the paths pre-authorized in the storage/dp/active/allowlist.txt.[1] Any divergence—whether an unauthorized file was touched or an authorized file was missed—must result in an immediate "STOP WORK" system failure state.
- **tools/lint/style.sh**: The style linter must enforce the structural rules of the new sidecar. As defined in the ledger history (archives/surfaces/SoP-2026-02-22-8cf576288.md), the linter must ensure that the six Mandatory Closing Block entries use distinct sentence structures, that the PR Description contains at least one markdown construct, and that the

Conversation Starter poses a genuine open question ending with a ?.[1]

6. Filing Doctrine and Toolchain Synchronization

Implementing the Strategic Path Forward requires precise modifications to the Stela environment's physical laws and directory structures. This ensures that the theoretical CbC framework maps exactly to the file system layout and definitions established in PoT.md Section 1.1.[1]

6.1 Exploiting the Storage Architecture

The physical laws defined in PoT.md classify the repository into distinct operational zones. The new Certified State Generation protocol heavily utilizes the separation between the storage/ tier and the var/tmp/ tier.[1]

- storage/dp/intake/: This remains the staging environment for the initial drafted packets.[1] DPs housed here must now structurally lack the static Mandatory Closing Block fields, reflecting the updated dp.md.tpl template.
- storage/handoff/: Defined as "Payload (handoff, dumps, dp)", this location currently holds the RESULTS and OPEN artifacts.[1] Under the new paradigm, it becomes the required, persistent domain for the active CLOSING-DP-OPS-XXXX.md sidecar during task execution.[1] The sidecar must remain here until consumed and destroyed by the certifier/prune sequence.
- var/tmp/: Designated exclusively for ephemeral worker scratch space ("Resume").[1] Because it is ephemeral, it must never be referenced as a canonical truth source or pointer target in any DP or ledger file. All persistent outputs must be routed to storage/ or archives/.
- archives/decisions/: As established in the typological split, this directory becomes the exclusive target for all outputs stemming from Investigative DPs.[1] To ensure structural uniformity and queryability, file names must follow the strict schema DEC-YYYY-MM-DD-<seq>-<slug>.md and include required YAML frontmatter (trace_id, decision_id, etc.).[1]

6.2 Toolchain Modification Blueprint

The executable layer within the ops/ directory must be updated to enforce the CbC methodology structurally, ensuring that operators cannot accidentally bypass the new generation constraints.[1]

Subsystem Element	Target Path	Required Structural	Strategic Rationale

		Modification	
DP Generator	ops/bin/draft	Strip all output logic that attempts to pre-fill the Mandatory Closing Block. Ensure the template renderer strictly generates the task framework but leaves execution outcomes completely undefined.	Eliminates the drafting-phase temporal paradox; physically prevents AI hallucination of final state.[1]
Certifier Engine	ops/bin/certify	Integrate sidecar ingestion logic. Implement programmatic git diff --name-only execution capture. Implement template rendering for the final RESULTS artifact based purely on actual runtime variables.	Enforces Just-In-Time (JIT) metadata generation; guarantees that the receipt proves the actual repository physical state.[1]
DP Linter	tools/lint/dp.sh	Add checks for sidecar presence during execution phase. Enforce YYYY-MM-DD literal string formatting for the Freshness Stamp. Ban command substitutions ($()) in the receipt block.	Detects procedural drift early; ensures strict compatibility with the certifier's literal replay engine, preventing execution failures.[1]

Integrator Prompts	docs/ops/prompts/e-prompt-03.md	Ban generative manifests. Enforce the Investigative/Deterministic typological split. Ban conditional If/Then execution paths.	Controls Lead AI behavior structurally through prompt architecture; establishes initial constraints before generation begins.[1]

6.3 Managing Context and Ledger Equilibrium

The final phase of the DP lifecycle requires updating the history ledger (SoP.md) and the proof ledger (PoW.md).[1] The Policy of Truth dictates that "SoP: history ledger only; no permanent rules live there" and "PoW: proof ledger only; execution evidence pointers live there".[1]

Under the new Certified State Generation framework, the Contractor AI does not manually generate or format the ledger entries during execution. The generation of PoW.md and SoP.md pointers is handled dynamically by ops/bin/certify at the moment of closeout.[1] The certifier emits schema-stamped surface leaves into the cold storage tier under archives/surfaces/ (e.g., archives/surfaces/PoW-2026-02-23-eb31f494c.md) and automatically rewrites the root PoW.md and SoP.md files to act as single-line HEAD pointers pointing to those newly generated leaves.[1]

This automated routing guarantees that the historical narrative (why a change occurred) and the execution evidence pointers (where the exact RESULTS and OPEN receipts live) are perfectly synchronized with the JIT-generated RESULTS receipt. This dynamic linking fulfills the core constitutional requirement that "everything the system proves is actually true," resolving the epistemic friction that previously plagued the manual ledger updates.[1]

7. Direct Application to Active Workstreams: The DP-OPS-0107 Case Study

To operationalize this Strategic Path Forward, it is necessary to apply the new architectural rules to a specific, real-world failure mode observed in recent DP drafting sessions. The intent of DP-OPS-0107 serves as an ideal case study.[1]

The original intent of DP-OPS-0107 was highly "mixed." The Integrator attempted to investigate the viability of a common.sh auto-invocation pattern while simultaneously attempting to execute a scaffold injection across multiple files based on the assumed results of that initial investigation.[1] This intent represents a textbook temporal paradox and a direct violation of the allowlist protocol. Because the specific files required for the scaffold injection could not be reliably predicted prior to the execution of the common.sh viability check, the Integrator was

forced into a hallucination loop to satisfy the drafting gates.

Under the new Certified State Generation protocol and typological bifurcation, DP-OPS-0107 must be ruthlessly split into sequential, deterministic operations.[1]

Step 1: Investigative DP Generation

The Integrator will draft a new packet, designated strictly as an Investigative DP.

- **Objective**: Verify the viability of common.sh auto-invocation within the current script hierarchy.
- **Allowlist**: archives/decisions/DEC-2026-02-23-002-common-viability.md. No operational shell scripts may be added to the allowlist, physically preventing code mutation.
- **Execution Plan**: The Contractor is instructed to execute specific tests, grep commands, or code audits against ops/lib/scripts/common.sh and related files to assess the hook architecture.[1]
- **Closeout**: Upon completion of the diagnostic checks, the Contractor authors the results and technical findings into the newly created Decision Record. The Operator runs ops/bin/certify, which dynamically reads the sidecar, executes a git diff --name-only, verifies that *only* the Decision Record was created, and compiles the RESULTS receipt.[1] The repository state remains pristine, but technical uncertainty is resolved.

Step 2: Operator Audit and Addendum Issuance

Following the successful completion and closeout of the Investigative DP, the Integrator performs a mandatory post-work audit, as defined in PoT.md Section 4.1.[1] The Integrator reviews the new Decision Record (DEC-2026-02-23-002-common-viability.md).

Based on the definitive findings within that record, the Integrator drafts an addendum recommendation using the ops/src/surfaces/addendum.md.tpl template. This addendum details the precise scaffold injection architecture required, now based on known facts rather than assumptions. The Operator reviews the recommendation, provides the OPERATOR_AUTHORIZATION field content, and signs the addendum.[1]

Step 3: Deterministic DP Generation

With the investigation concluded and the findings crystallized into a canonical Decision Record, the Integrator is now equipped to draft the second packet (e.g., DP-OPS-0108).

- **Objective**: Implement scaffold injection according to the architecture authorized in Decision Record DEC-2026-02-23-002-common-viability.md.
- **Allowlist**: A highly specific, definitive list of files to be modified (e.g., ops/bin/scaffold, docs/ops/specs/binaries/scaffold.md, and specific factory definition files) is provided.
- **Execution Plan**: The Contractor follows a linear, imperative, non-branching set of instructions to modify the exact files listed in the allowlist. No If/Then logic is present.

- **Closeout**: The Contractor completes the modifications and updates the CLOSING-DP-OPS-0108.md sidecar file with the descriptive PR details. The Operator runs ops/bin/certify to dynamically generate the real-time manifest, confirm it matches the strict allowlist, and finalize the transaction.[1]

This phased, deterministic approach entirely eliminates the need for the Integrator to predict file impacts, preserving the structural integrity of the PoT.md mandates and maintaining the repository in a state of continuous, mathematically provable validity.

8. Third-Order Implications and System Resilience

The transition to Certified State Generation produces several vital third-order effects that significantly enhance the resilience and scalability of the Stela platform's governance model.

First, by removing the burden of perfect prediction from the Integrator AI, the system drastically reduces token overhead and context window exhaustion during the drafting phase. The Integrator's computational focus is redirected toward optimizing the sequential logic of the Execution Plan rather than guessing Git diff outputs. This improves the reliability of the resulting task instructions.

Second, the JIT generation of the RESULTS manifest establishes an unimpeachable audit trail. External auditors or future Integrator agents reviewing the PoW.md ledger can possess absolute confidence that the file paths listed in a DP's receipt were physically modified by that exact operation, because the manifest was generated by the kernel (ops/bin/certify) reading the filesystem, not by an LLM generating text.[1] This fulfills the epistemic standard that "Truth is a primitive predicate governed by specific rules rather than a mere definition".[1]

Finally, the strict bifurcation into Investigative and Deterministic packets formalizes a scientific method within repository operations. Hypothesis testing (Investigation) is structurally separated from engineering (Implementation). This prevents experimental code from polluting the main branch trunk, ensuring that the main branch remains a pristine, verified state at all times, in accordance with the Branching Doctrine (SSoT).[1]

9. Conclusion: The Definitive Resolution

The temporal paradox observed in the Stela repository's legacy drafting process represents a critical threat to the systemic equilibrium established by the Policy of Truth. By forcing Integrator agents to hypothesize downstream execution states, the system inadvertently mandated the fabrication of evidence, directly violating the Generation Mandate and generating cascading linting failures due to Canon Drift and Ambiguity.

The single Strategic Path Forward is the comprehensive implementation of Certified State Generation bounded by Correct-by-Construction methodologies. By bifurcating task typologies into strictly Investigative workflows (producing only Decision Records) or strictly

Deterministic workflows (producing specific code mutations), the system eliminates branching uncertainty.[1] By decoupling the Mandatory Closing Block from the pre-execution DP template and relocating it to a real-time sidecar (storage/handoff/CLOSING-DP-OPS-XXXX.md), the system anchors truth in observable reality rather than prediction.[1] Ultimately, by elevating the ops/bin/certify binary to dynamically ingest the sidecar and generate the technical manifest via immediate git diff execution at the moment of closeout, the system mathematically guarantees that metadata is an absolute, immutable reflection of the repository's physical state.[1]

This structural transition physically prevents prediction errors, aligning perfectly with the constitutional axiom that everything the system proves is actually true.[1] Execution of Phase 0 through Phase 3 will immediately restore functional integrity, allowing the Operator, Integrator, and Contractor to interact within a deterministic, reality-based continuous integration cycle, free from the destabilizing influence of necessary hallucination.

Works cited

1. dump-full-main-7567d65ae.txt
2. Correct by Construction - Wikipedia, accessed February 23, 2026, https://en.wikipedia.org/wiki/Correct_by_Construction
3. Correct by Construction Correct by Construction - Nate Rush, accessed February 23, 2026, https://www.naterush.io/posts/2018-12-31-correct-by-construction-correct-by-construction.markdown
4. eBook | Winning the PCB Trifecta via "Correct by Construction" Design Principles - Cadence, accessed February 23, 2026, https://www5.cadence.com/Correct_by_Construction_ebook.html